SC3020-CE4301-CZ4031: Project 2

Prepared by Yufei Tao
15 Oct, 2025

DEADLINE: 11:59pm, 20 Nov, 2025.

1 Main Objective

This project is designed to deepen your understanding of query optimization and concurrency control through authentic experience on a real database system.

2 Setup

This guide assumes that you are using Windows, which is the operating system installed on all the machines in the CCDS labs.

First, download PostgreSQL at `https://www.postgresql.org/download` and install it. Make sure `pgAdmin4` is working — this is a utility program that allows you to access and manage the database with a friendly GUI.

Second, download the file `proj2-data.zip` from NTULearn (look for `Project 2` and then `Data for Project 2`). Unzip the file and you should see 8 `csv` files. These files were generated according to the TPC-H benchmark (see `https://www.tpc.org/tpch` for the details of TPC-H).

Third, download the file `table-creation.txt` from NTULearn (look for `Project 2` and then `Table Creation Statements`). Use `pgAdmin4` to create a new database in PostgreSQL, and then use the statements in `table-creation.txt` to create 9 tables inside that database. Make sure that you see the following tables in `pgAdmin4`:

- `customer`
- `lineitem`
- `nation`
- `orders`
- `part`
- `partsupp`
- `region`
- `supplier`
- `t`

Fourth, use `pgAdmin4` to import data into the first 8 tables from the `csv` files you obtained earlier. The file names match the table names in a straightforward manner (e.g., file `customer.csv` is for table `customer`). Note the foreign key constraints on the tables; choose a table ordering to do the importing by respecting those constraints.

3 Tasks

3.1 Query Optimization

Familiarize yourself with the use of `explain` and `analyze` in PostgreSQL:

- Putting `explain` before an SQL query prompts the system to explain the execution plan selected for the query. For example, try:
  `explain select * from lineitem;`
  to see what happens.
- Putting `explain analyze` before an SQL query prompts the system to (i) explain the execution plan selected for the query and (ii) physically execute the plan to obtain the actual performance statistics. For example, try:
  `explain analyze select * from lineitem;`
  to see what happens.

3.1.1 Task 1

The `d+` command allows you to find the basic information of each table. Figure out the syntax of `d+` from online sources. Identify the index that PostgreSQL creates on each table automatically.

3.1.2 Task 2

Boot up your system and run the following as your first query.

```
explain analyze select * from lineitem where l_quantity >= 50;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Run the query for a second time. Does it "feel" faster? How much faster? Why did it happen?

3.1.3 Task 3

Create an index with the following statement:

```
create index idx_lineitem_quantity on lineitem (l_quantity);
```

Use `d+` to find out what index it is. Now run again

```
explain analyze select * from lineitem where l_quantity >= 50;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan chosen in Task 2? Why?

Before proceeding to the next task, use the following statement to remove the index.

```
drop index idx_lineitem_quantity;
```

3.1.4 Task 4

Create an index with the following statement:

```
create index idx_lineitem_quantity_hash on lineitem using hash (l_quantity);
```

Now run

```
explain analyze select * from lineitem where l_quantity >= 50;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan chosen in Task 3? Why?

Now run instead

```
explain analyze select * from lineitem where l_quantity = 50;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan chosen for the previous query? Why?

Before proceeding to the next task, use the following statement to remove the index.

```
drop index idx_lineitem_quantity_hash;
```

3.1.5 Task 5

Execute the following command:

```
set work_mem = '1MB';
```

Now run

```
explain analyze select l_orderkey, l_suppkey, sum(l_quantity) as sum_qty
from lineitem
group by l_suppkey, l_orderkey
order by l_suppkey, l_orderkey;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query.

Now, execute the following command:

```
set work_mem = '1000MB';
```

Run the same query again and explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from the plan used for 1MB of working memory? Why?

3.1.6 Task 6

Execute the following command:

```
set work_mem = '1MB';
```

Now run

```
explain analyze select * from lineitem l1, lineitem l2
where l1.l_orderkey = l2.l_suppkey;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. To do so, you need to learn from online sources the meaning of a *batch* (this is a concept specific to the hash join implementation of PostgreSQL).

Now, execute the following command:

```
set work_mem = '64MB';
```

Run the same query again and explain how the query plan has changed? Why did this happen?

3.1.7 Task 7

Execute the following command:

```
set work_mem = '1MB';
```

Now run

```
explain analyze select * from lineitem l1, lineitem l2
where l1.l_orderkey = l2.l_suppkey
order by l2.l_suppkey;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Is it different from what you saw in Task 6? Why?

Now, execute the following command:

```
set work_mem = '1000MB';
```

Run the same query again and explain how the query plan has changed. Why did this happen?

Now, execute the following command:

```
set work_mem = '2000MB';
```

Run the same query for a third time and explain how the query plan has changed. Why did this happen?

3.1.8 Task 8

Execute the following command:

```
set work_mem = '1MB';
```

Now run:

```
explain analyze select * from lineitem l, orders o
where l.l_orderkey = o.o_orderkey order by o.o_orderkey;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Was any sorting performed? Why did this happen?

If it helps, you can use the following command to see how the tuples of a relation (here we use `orders` as an example) are physically ordered on disk:

```
select ctid, * from orders LIMIT 100;
```

3.1.9 Task 9

Execute the following commands:

```
set work_mem = '1MB';
create index idx_lineitem_quantity on lineitem (l_quantity);
```

Now run:

```
explain analyze select * from lineitem l, orders o
where l.l_partkey = o.o_custkey and l.l_quantity = 20
order by o.o_custkey;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Did the plan use the index `idx_lineitem_quantity` we built? How do you explain this?

Now run instead:

```
explain analyze select * from lineitem l, orders o
where l.l_partkey = o.o_custkey and l.l_orderkey <= 10000
order by o.o_custkey;
```

Explain in plain words the query plan that PostgreSQL selects to answer the query. Did the plan use any indexes? How do you explain this?

Before finishing this task, execute the command below:

```
drop index idx_lineitem_quantity;
```

3.1.10 Task 10

Run:

```
explain analyze select * from lineitem l1, lineitem l2
where l1.l_orderkey = l2.l_orderkey;
```

What was the result size predicted by the query planner? What is the actual result size?

Run the following three queries:

- `select count(*) from lineitem;`
- ```
  select count(*) from lineitem l1, lineitem l2
  where l1.l_orderkey = l2.l_orderkey;
  ```
- ```
  select count(*) from lineitem l1, lineitem l2, lineitem l3
  where l1.l_orderkey = l2.l_orderkey and l2.l_orderkey = l3.l_orderkey;
  ```

How do you explain the running time differences of the above queries? By the way, you can terminate the 3rd query if it fails to terminate within 5 minutes.

If you can change the implementation of PostgreSQL, do you see an algorithm that can process all three queries with negligible cost differences?

Rewrite the 3rd query into another SQL query that returns the same result in less than 30 seconds (hint: `group by` and nested SQL).

3.2 Transactions

3.2.1 Task 11

First, execute the command:

```
insert into public.t values(1, 10);
```

Now, start a transaction:

```
START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Run the following query, which we will refer to as $Q1$:

```
select * from public.t;
```

Open *another* command-line session (in pgAdmin4, go to menu `Tools` and then click on `PSQL Tool`). In this session, execute the following commands:

```
START TRANSACTION ISOLATION LEVEL SERIALIZABLE;
insert into public.t values(2, 20);
COMMIT;
```

Now, go back to the first command-line session, and execute the following query, which we will refer to as $Q2$:

```
select * from public.t;
```

Are the results of $Q1$ and $Q2$ identical? What anomalies — according to the lecture slides "Transactions 2: Isolation Levels" — have you witnessed? Why is this allowed?

Before proceeding, close both command-line sessions.

3.2.2 Task 12

Open a new command-line session and execute the command:

```
delete from public.T where id >= 2;
```

Start a transaction — which we denote as T_1 — with:

```
START TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Run the following query:

```
select * from public.t;
```

Open *another* command-line session and execute the following commands:

```
START TRANSACTION ISOLATION LEVEL SERIALIZABLE;
insert into public.t values(2, 20);
COMMIT;
```

Now, go back to the command-line session of T_1 and execute:

```
insert into public.t values(3, 30);
```

What do you observe? Why does this happen?

Before proceeding, close both command-line sessions.

3.2.3 Task 13

Open a new command-line session and execute the commands:

```
delete from public.T where id >= 2;
insert into public.t values(1, 10);
```

If the second command reports a primary key constraint error, it means that the tuple (1, 10) already exists, in which case all is good and you may proceed. Start a transaction — which we denote as T_1 — with:

```
START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Open *another* command-line session and start a transaction — which we denote as T_2 — with:

```
START TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Go to the session of T_1 and execute:

```
update public.T set v = 20 where id = 1;
```

Now, go to the session of T_2 and execute:

```
update public.T set v = 30 where id = 1;
```

What do you observe? Why does this happen?

Return to the session of T_1 and execute:

```
COMMIT;
```

Switch back to the session of T_2. What do you observe? Why does it happen?

In the session of T_2, execute:

```
select * from public.t;
```

What do you see? Now execute the following commands:

```
ROLLBACK;
select * from public.t;
```

What do you see? Why does it happen?

4 What to Submit

Each project group should submit a report in the pdf format. The report must contain a separate section for every task. Each section should include

- the necessary screenshots to demonstrate how the corresponding task was carried out;

- convincing explanations for your answers to the questions asked in this document. As much as possible, relate your explanations to our lecture slides and base your explanations on hard evidence from the query plan details.

In addition, the report must include a declaration about each group member's contributions, including a contribution percentage for each member (e.g., if your group has 5 members and everyone has contributed equally, you should put down 20% for each person).

One submission per group is sufficient. The submission deadline is **11:59pm, 20 Nov, 2025**.